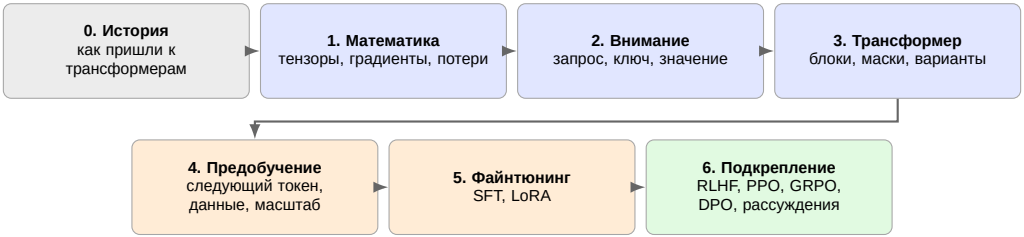
Языковые модели: от матриц до рассуждений

О. М. Атаева А. П. Халов

Осень 2026

Курс «От интеллектуального поиска к интеллектуальным агентам», ВМК МГУ

План



Синие блоки отвечают на вопрос «как устроена модель», оранжевые на вопрос «как её учат», зелёный на вопрос «как её учат быть полезной и рассуждать».

Живые примеры посчитаны на маленькой открытой модели Qwen2.5-0.5B: 494 млн параметров, 24 слоя, 14 голов внимания, словарь 151 936 токенов. Модель только считала вероятности, её никто не обучал.

Главное

Языковая модель есть функция, которая по тексту выдаёт вероятности следующего токена. Всё остальное отвечает на вопросы, как эту функцию устроить и как подобрать её веса.

Семьдесят лет до ChatGPT

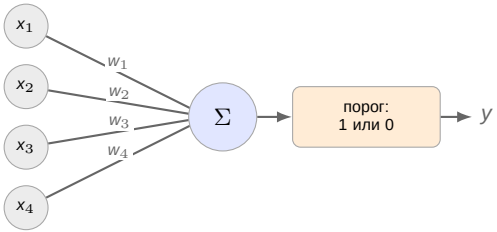


Два открытия этого пути отмечены Нобелевской премией по физике 2024 года: её получили Джон Хопфилд и Джеффри Хинтон за основополагающие работы о нейронных сетях. Хинтон был одним из авторов метода обратного распространения (Румельхарт, Хинтон, Уильямс, 1986).

Главное

Каждая веха решала одну задачу: как моделировать нейрон, как обучать сеть, как помнить контекст, как обрабатывать длинный текст параллельно.

Перцептрон: модель нейрона



Фрэнк Розенблатт, 1957. На вход поступают числа $x_1, \ldots, x_n$. Каждое умножается на свой вес, всё складывается:

$$s = \sum_i w_i x_i = \mathbf{w} \cdot \mathbf{x}.$$

Если сумма больше порога, на выходе 1, иначе 0. Порог добавляет нелинейность.

Веса подбираются по примерам. В этом и состоит обучение.

Главное

Сумма произведений входов на веса есть скалярное произведение. Любая современная нейросеть состоит из огромного числа таких операций.

От рекуррентных сетей к трансформеру

Рекуррентная сеть читает текст по токену и несёт память в скрытом состоянии. Память простая, и о начале длинного текста сеть забывает.

LSTM (Хохрайтер, Шмидхубер, 1997) добавила механизмы забывания, записи и удержания. LSTM стала основой нейронных языковых моделей и машинного перевода 2010-х.

AlexNet (Крижевский, Суцкевер, Хинтон, 2012) показала, что глубокие сети становятся лучше с ростом данных и что учить их надо на GPU.

Внимание (Бахданау, Чо, Бенджио, 2014) позволило переводчику на каждом шаге смотреть на нужные слова исходного предложения.

Трансформер («Attention Is All You Need», Васвани и соавторы, 2017) оставил одно внимание и убрал рекуррентность. Итог:

- длинные тексты обрабатываются лучше;
- все токены обрабатываются параллельно, обучение быстрее;
- качество выросло на переводе, классификации, генерации.

Главное

Трансформер убрал последовательное чтение. Это сделало возможным обучение на огромных корпусах.

GPT-3, ChatGPT, o1: три поворота

2020: GPT-3

175 млрд параметров, примерно в 10 раз больше прежних крупнейших моделей. Модель решала новые задачи по нескольким примерам прямо в запросе, без дообучения. Это назвали обучением в контексте.

2022: ChatGPT

Первый массовый чат-бот на языковой модели. Задумывался как демонстрация возможностей и стал началом бума. Модель дообучена следовать инструкциям и учитывать предпочтения людей.

2024: o1

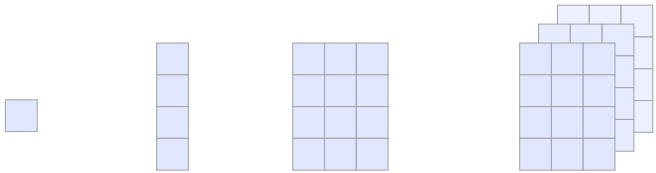
Модель рассуждает перед ответом. Чем дольше рассуждение, тем выше качество. Вторая ось роста: вычисления во время ответа.

ось 1: больше данных и параметров при обучении + **ось 2**: больше рассуждений при ответе

Главное

Модели умнеют за счёт двух вещей: масштаба обучения и длины рассуждения во время работы.

Скаляр, вектор, матрица, тензор



скаляр
одно число

вектор
столбец чисел, длина n

матрица
таблица $M \times N$, индексы строка и столбец

тензор
много измерений: 3D, 4D

Вектор есть частный случай матрицы, матрица есть частный случай тензора. Тензор третьего порядка есть набор матриц.

Пример тензора в языковой модели. Пакет из 2 предложений по 3 токена, каждый токен есть вектор из 4 чисел: тензор $2 \times 3 \times 4$.

Главное

Нейросеть есть последовательность операций над тензорами. Самая частая операция есть умножение.

Скалярное произведение и умножение матриц

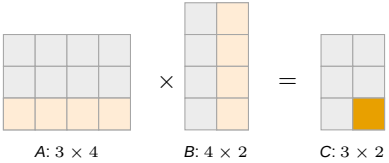
Скалярное произведение двух векторов одной длины: перемножить элементы с одинаковыми номерами и сложить.

$$\mathbf{a}\cdot\mathbf{b} = \sum_{i=1}^{n} a_i b_i, \qquad (1,2,3)\cdot(4,0,-1) = 4+0-3 = 1.$$

Это ровно то, что делает перцептрон.

Произведение матриц: элемент (i, j) есть скалярное произведение i-й строки первой матрицы на j-й столбец второй.

$$A_{m\times k} \cdot B_{k\times n} = C_{m\times n}.$$



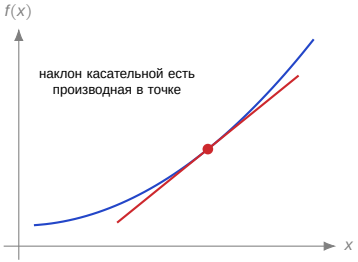
Элемент $C_{3,2}$: третья строка A на второй столбец B. Число столбцов A должно совпасть с числом строк B.

Тензор $2 \times 3 \times 4$ на матрицу весов 4×2 даёт $2 \times 3 \times 2$: каждая подматрица умножается отдельно.

Главное

Модель с сотнями миллиардов параметров есть огромный набор матриц, которые перемножаются при каждом токене. Для этого и нужны GPU.

Производная, градиент и обучение как оптимизация



Производная показывает, как быстро функция меняется в точке.

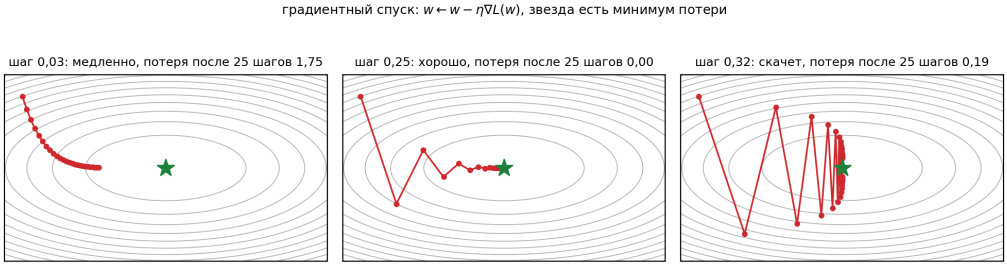
Модель тоже функция: вход есть текст, параметры есть веса, выход есть вероятности. Ответ сравнивают с правильным и считают ошибку, она же функция потерь L.

Градиент ∇L есть вектор частных производных по всем весам. Он показывает направление скорейшего роста потери. Градиент 1,5 по w_1 и 0,1 по w_2 означает, что потеря в 15 раз чувствительнее к w_1.

Главное

Обучение есть подбор весов, при которых потеря минимальна. Для этого нужен градиент.

Градиентный спуск



$$w \leftarrow w - \eta \nabla L(w)$$

Посчитать градиент потери, шагнуть в обратную сторону, повторить. Шаг обучения η (learning rate) решает судьбу обучения: малый шаг идёт медленно, удачный быстро сходится, большой скачет поперёк долины, слишком большой уводит прочь от минимума.

Главное

Градиентный спуск есть единственный механизм обучения всех нейросетей. Различаются только потеря и способ считать градиент.

Токены и кросс-энтропия

Токен есть единица текста в словаре модели: часть слова, слово, знак. Словарь Qwen2.5 содержит 151 936 токенов. Разбиение у каждой модели своё:

текст	токенов	как разрезан
дом	2	д · ом
нейросеть	3	ней · рос · еть
house	1	house
Гарри Поттер взял волшебную палочку	18	Г · ар · ри · П · от · тер · вз · я · л · …
Harry Potter took the magic wand	6	Harry · Potter · took · the · magic · wand

Токенизатор Qwen2.5. Русский текст обходится этой модели втрое дороже английского.

Задача модели: предсказать следующий токен. Модель выдаёт вероятность p_i каждому токену словаря. В данных у правильного токена вероятность 1, у остальных 0.

Кросс-энтропия меряет, насколько предсказанное распределение далеко от истинного:

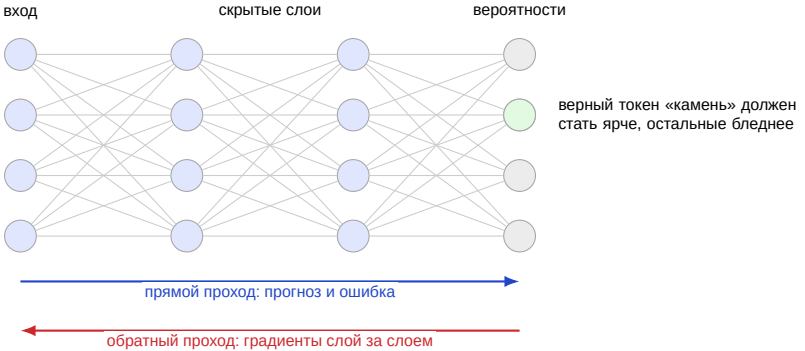
$$L = -\sum_{i=1}^{|V|} y_i \log p_i = -\log p_{\text{верный}}.$$

Если модель дала верному токену 0,85, потеря $-\ln 0{,}85 \approx 0{,}16$. Если 0,01, потеря 4,6.

Главное

Языковая модель учится одному: давать большую вероятность настоящему следующему токену.

Обратное распространение ошибки



Пример: вход «Гарри Поттер и философский …», верный следующий токен «камень». Активации уже посчитаны, менять можно только веса. Градиент показывает, какие нейроны предыдущего слоя сильнее всего повлияли на ответ, и их веса меняются сильнее. Так ошибка идёт от выхода ко входу, и на каждом слое градиент считается на месте по правилу производной сложной функции.

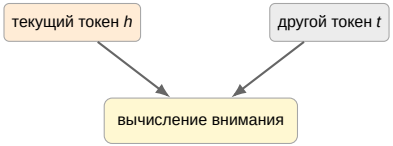
Главное

Обратное распространение есть способ быстро посчитать градиент по миллиардам весов. Математически это цепное правило из школьного курса.

Зачем внимание

Рекуррентная сеть читает слово за словом, и память о начале текста слабеет. Человек читает иначе: выделяет важные слова и связывает их через весь текст.

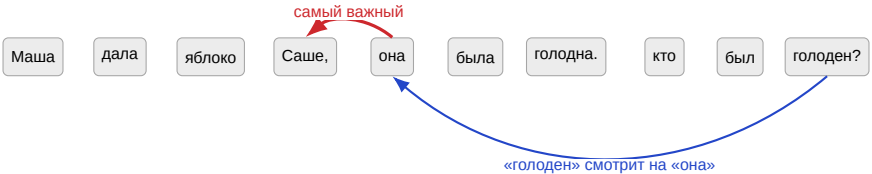
Внимание есть слой нейросети, который на каждом шаге решает, какие части входа сейчас важнее. **Самовнимание** взвешивает все токены последовательности относительно друг друга.



Главное

Внимание даёт сети смотреть на текст целиком и находить связи между словами на любом расстоянии.

Пример: кто был голоден?



Модель получает текст «Маша дала яблоко Саше, она была голодна» и вопрос «Кто был голоден?». Для токена «она» самым важным оказывается «Саша». Для «голоден» самым важным оказывается «она». Цепочка связей даёт ответ.

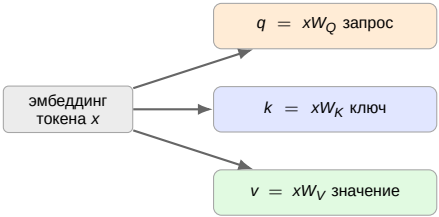
Внимание считается для каждой пары токенов. Это сила метода и его цена:

токенов в контексте	1 000	8 000	128 000
пар для внимания	1 млн	64 млн	16,4 млрд

Главное

Вычисления внимания растут квадратично с длиной контекста. Поэтому длинный контекст дорог.

Запрос, ключ, значение



- **Запрос**: чем токен «спрашивает» другие: «есть ли у вас что-то, связанное со мной?»
- **Ключ**: что токен может предложить. Ключ говорит, есть ли нужная информация и сколько.
- **Значение**: сама информация, которую токен отдаёт, если его сочли важным.

Матрицы W_Q, W_K, W_V обучаемые.

Эмбеддинг есть числовой вектор токена. Похожие по смыслу токены получают похожие эмбеддинги.

Главное

Скалярное произведение запроса одного токена на ключ другого даёт вес внимания. Значения складываются с этими весами.

Формула внимания на учебных числах

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Учебный пример, $d_k = 2$. Запрос токена «она»: $q = (2, 1)$. Ключи:

токен	ключ k	$q \cdot k$	$/\sqrt{2}$	softmax
Маша	$(1, 0)$	2	1,41	0,31
яблоко	$(0, 0)$	0	0	0,07
Саше	$(1, 1)$	3	2,12	0,62

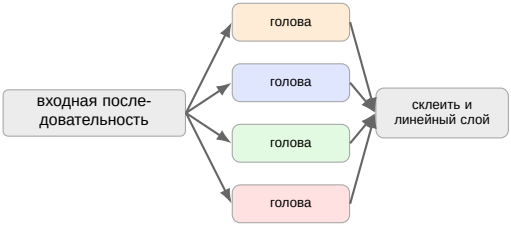
Числа учебные, выбраны для наглядности.

1. Скалярные произведения запроса на ключи.
2. Деление на $\sqrt{d_k}$: без него при большой размерности произведения велики, и softmax вырождается в одну единицу.
3. Softmax превращает числа в веса от 0 до 1 с суммой 1.
4. Выход есть $0{,}31\, v_{\text{Маша}} + 0{,}07\, v_{\text{яблоко}} + 0{,}62\, v_{\text{Саше}}$.

Главное

Новый вектор токена есть смесь значений всех токенов, и больше всего в смеси того, на что токен «смотрел».

Многоголовое внимание



Внимание считается несколько раз параллельно, каждая голова со своими матрицами W_Q, W_K, W_V. В исходном трансформере голов 8, в Qwen2.5-0.5B их 14.

Одно слово связано с другими многими отношениями: грамматическими, смысловыми, по ссылке. Одна голова обычно ловит одно отношение, несколько голов ловят несколько.

Главное

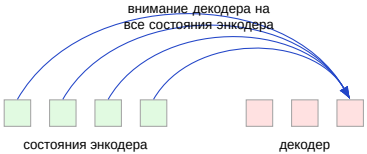
Многоголовое внимание смотрит на текст сразу с нескольких сторон. Выходы голов склеиваются и проходят через линейный слой.

Энкодер и декодер

Энкодер обрабатывает входную последовательность, например запрос.

Декодер генерирует ответ по одному токену, начиная со служебного токена начала <bos>. Каждый сгенерированный токен добавляется в контекст.

До трансформера энкодер и декодер были рекуррентными, и внимание связывало только декодер с энкодером.

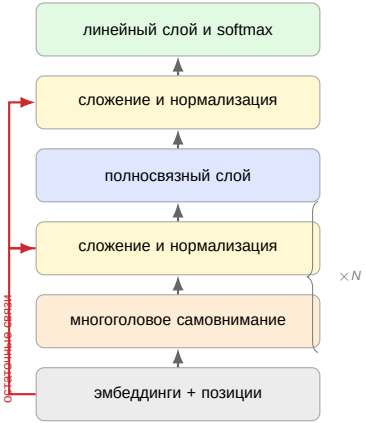


В трансформере рекуррентности нет совсем: внимание заменяет последовательную обработку и в энкодере, и в декодере. Все токены обрабатываются параллельно. Отсюда название статьи: одного внимания достаточно.

Главное

Трансформер оставил схему энкодер–декодер и заменил рекуррентность вниманием.

Блок трансформера



- **Самовнимание**: каждый токен смотрит на все остальные и собирает контекст.
- **Полносвязный слой**: вектор каждого токена отдельно проходит через небольшую сеть, сеть «обдумывает» собранное.
- **Остаточные связи**: вход блока прибавляется к его выходу. Исходная информация не теряется в глубокой сети.
- **Нормализация слоя**: из вектора вычитается среднее, он делится на стандартное отклонение, затем масштабируется обучаемыми параметрами. Это стабилизирует обучение.
- Блок повторяется N раз: в Qwen2.5-0.5B 24 раза.

Главное

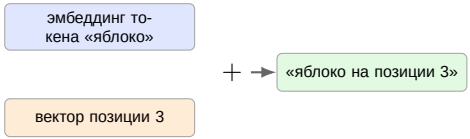
Трансформер есть стопка одинаковых блоков «внимание, затем полносвязный слой» с остаточными связями и нормализацией.

Эмбеддинги и позиция токена

Модель принимает только числа, поэтому каждый токен заменяется вектором-эмбеддингом из таблицы. В Qwen2.5-0.5B вектор имеет 896 чисел, таблица 151 936 строк.

Трансформер обрабатывает все токены одновременно и сам по себе порядка не видит. «Маша дала Саше» и «Саша дала Маше» дали бы один набор векторов.

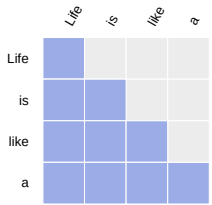
Поэтому к эмбеддингу прибавляется **позиционное кодирование**: вектор, который зависит от номера позиции.



Главное

Смысл токена даёт эмбеддинг, место в тексте даёт позиционное кодирование. Модель получает их сумму.

Маска: не подглядывать в будущее



Строка видит себя и токены левее. Серые клетки закрыты.

При обучении декодеру подают всю последовательность сразу. Для каждого токена он должен предсказать следующий. Если ничего не делать, он увидит ответ: следующий токен уже стоит во входе.

Маска закрывает каждому токену доступ к токенам правее: их веса внимания обнуляются. Пока сгенерировано «Life», внимание на «is», «like», «a» закрыто.

Сдвиг вправо (shifted right) означает, что вход декодера сдвинут на один токен: на позиции i предсказывается токен $i + 1$.

Главное

Маска позволяет учить модель на всех позициях текста параллельно, и при этом каждая позиция видит только прошлое.

Softmax: из чисел в вероятности

Внимание и выход модели дают произвольные числа: положительные, отрицательные, большие, малые. Softmax превращает их в распределение вероятностей:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Все значения от 0 до 1, сумма равна 1, порядок сохраняется, большие числа выделяются сильнее.

	токен 1	токен 2	токен 3	токен 4
числа z	3,0	1,0	0,2	$-1,0$
e^z	20,09	2,72	1,22	0,37
softmax	0,82	0,11	0,05	0,02

Сумма e^z равна 24,40. Отрицательное число даёт малую, но положительную вероятность.

Главное

Softmax стоит дважды: в каждом внимании и на выходе модели, где он даёт вероятности следующего токена.

Варианты архитектуры

только декодер

Энкодер отброшен, вся модель есть декодер с маской. Проще и быстрее учится. Так устроены GPT, Qwen, Llama и большинство чат-моделей.

смесь экспертов

Вместо одного полносвязного слоя несколько «экспертов». Каждый токен обрабатывают только доверенные ему эксперты, их ответы складываются. Внимание общее.

мультимодальный

Свой энкодер на каждую модальность: картинки, звук, видео. Их эмбеддинги отображаются в общее пространство с текстом и склеиваются на входе модели.

Во всех трёх вариантах блок «внимание, полносвязный слой» остаётся тем же. Меняется то, что стоит вокруг него.

Главное

Современная чат-модель есть трансформер из одного декодера. Смесь экспертов экономит вычисления, мультимодальность добавляет энкодеры.

Предсказание следующего токена

1. Текст разбивается на токены, токенам сопоставляются номера.
2. Токены режутся на куски длиной в **контекстное окно**, например 512. У современных моделей окно достигает сотен тысяч токенов.
3. Модель на каждой позиции предсказывает следующий токен. Маска закрывает будущее.
4. Ошибка считается кросс-энтропией, веса обновляются обратным распространением.

что модель видит	что предсказывает
Гарри	Поттер
Гарри Поттер	взял
Гарри Поттер взял	волшебную
Гарри Поттер взял волшебную	палочку

Один кусок текста даёт столько учебных примеров, сколько в нём токенов.

Главное

Предобучение не требует разметки: правильный ответ есть сам следующий токен текста.

Вероятности следующего токена и температура



Настоящие вероятности модели Qwen2.5-0.5B для продолжения «Harry Potter raised his magic». При температуре 1 токен «wand» получает 0,85, при 0,5 почти 1, при 1,5 только 0,26. Температура делит числа на выходе перед softmax: низкая делает выбор почти детерминированным, высокая разнообразным.

Главное

Модель не выбирает слово, она даёт распределение. Как из него брать токен, решает температура: низкая для точных ответов, высокая для творческих.

Пакет, итерация, эпоха

Пакет (батч): несколько окон сразу, например 32 окна по 512 токенов.

Итерация: один прямой и один обратный проход пакета, один шаг градиентного спуска.

Эпоха: модель увидела весь корпус один раз.

Пример счёта: корпус из 1 млн окон, пакет 32 окна. Одна эпоха есть $1\,000\,000/32 = 31\,250$ итераций.

Компромиссы

- слишком большой пакет не помещается в память, слишком малый неэффективно использует GPU;
- мало эпох: модель недообучена, много эпох: переобучена.

Размеры подбираются опытом.

Главное

Обучение есть многократный проход по корпусу пакетами, и каждая итерация чуть сдвигает веса.

Данные для предобучения

Откуда

- архив веб-страниц Common Crawl, новости, блоги, форумы;
- книги и научные статьи;
- Википедия;
- код, расшифровки видео и другое.

Самообучение (self-supervised): разметка не нужна, поэтому доступны миллиарды текстов.

Как чистят

- удаление дубликатов: повторы ведут к запоминанию;
- фильтрация мусора: реклама, сбои кодировки;
- отбор по языку, теме и качеству, часто специальными классификаторами;
- удаление оскорблений и неэтичного содержания.

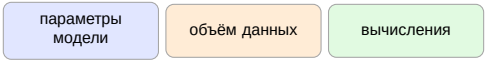
Модель перенимает всё, что есть в данных: ошибки, стереотипы, перекосы. Ей никто не показывает, что правильно.

Главное

Сбор и очистка данных есть самая важная часть предобучения. Модель знает то, что было в её корпусе.

Законы масштабирования

До 2020 года считалось, что большие модели начнут переобучаться и перестанут улучшаться. Работа OpenAI «Scaling Laws for Neural Language Models» (Каплан и соавторы, 2020) показала иное: качество предсказуемо растёт с тремя величинами.



Зависимость описывается степенными законами: потеря падает плавно и предсказуемо при росте каждой величины.

Цена масштаба

- качественные открытые тексты кончаются, синтетические данные менее полезны;
- обучение требует кластеров из тысяч GPU, это миллиарды долларов и большой расход энергии;
- большую модель дорого и запускать: нужна память и вычисления даже для ответа.

Главное

Масштаб даёт качество предсказуемо, и масштаб дорог. Это толкнуло к поиску второй оси роста: рассуждений во время ответа.

Зачем дообучать

После предобучения модель знает язык и закономерности текстов. Следовать инструкциям, решать задачи, вести себя определённым образом её никто не учил.

Файнтюнинг: взять предобученную модель и дообучить на небольшом специальном наборе примеров нужных задач.

Кто использует

- **производители моделей**: улучшить программирование, математику, сделать модель безопаснее (выравнивание);
- **разработчики**: подстроить модель под свою область без обучения с нуля. Это возможно с открытыми моделями, веса которых выложены, или через специальный интерфейс поставщика.

Главное

Предобучение даёт знание языка, файнтюнинг делает модель полезной для конкретной работы.

Два вида дообучения

без разметки

Дообучение на сырых текстах новой области тем же предсказанием следующего токена. Пример: модель видела мало математики, её дообучают на учебниках. Или на внутренней документации компании. Годится, когда размеченных данных нет.

с разметкой (SFT)

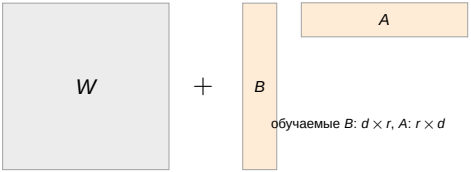
Дообучение на парах «вход, желаемый выход»: инструкция и ответ, вопрос и решение. Данные готовить сложнее, их делают люди. SFT учит следовать инструкциям и точно решать задачи. Чаще всего идёт сразу после предобучения.

```
{"instruction": "Объясни, что делает LoRA, в одном предложении.",
 "output": "LoRA добавляет к замороженным весам модели маленькие обучаемые матрицы-адаптеры."}
```

Главное

Без разметки модель узнаёт новую область, с разметкой учится нужному поведению.

Полный файнтюнинг и LoRA



$$W' = W + BA$$

Полный файнтюнинг меняет все веса. Надёжно и дорого.

LoRA (Ху и соавторы, 2021) замораживает исходные веса и учит малоранговую поправку BA с маленьким рангом r.

матрица 4096×4096	параметров
полный файнтюнинг	16 777 216
LoRA, $r = 8$: $2 \cdot 4096 \cdot 8$	65 536

В 256 раз меньше обучаемых параметров. QLoRA (Деттмерс и соавторы, 2023) хранит замороженные веса в 4 битах.

Главное

LoRA и её варианты есть стандарт дообучения при ограниченных ресурсах: мало параметров, мало памяти, быстрая сходимость.

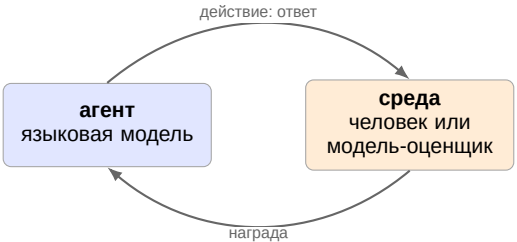
Рецепт дообучения своей модели

1. **Задача и данные.** Пары вопрос–ответ: своя разметка или готовый набор с Hugging Face.
2. **Модель.** Открытая модель под свою GPU: дома до 8 млрд параметров.
3. **Окружение.** Доступ к GPU, например Google Colab.
4. **Модель и токенизатор.** Загрузка в 4 битах для экономии памяти, токен заполнения для пакетов.
5. **Данные в формат модели.** Обернуть каждую пару в служебные токены, например `[INST] … [/INST]`.
6. **Адаптер LoRA.** Ранг r и слои, куда встраивается адаптер, например проекции запроса и значения.
7. **Обучение.** Число эпох, размер пакета, куда сохранять.
8. **Проверка.** Загрузить базовую модель так же, подключить адаптер, задать вопросы.

Главное

Файнтюнинг доступен дома: в отличие от предобучения, ему хватает одной GPU и тысяч примеров.

Обучение с подкреплением



Файнтюнинг учит воспроизводить готовые ответы. Он не говорит модели, какие ответы на самом деле хороши для человека.

В обучении с подкреплением модель получает **награду** за свой ответ и меняет поведение, свою **политику**, чтобы награды росли.

Пример. Вопрос о столице Франции. Модель отвечает «Марсель». Оценщик даёт отрицательную награду. Модель меняет веса так, чтобы в следующий раз с большей вероятностью ответить «Париж».

Главное

Подкрепление учит модель на оценке её собственных ответов. Файнтюнинг учил на чужих готовых ответах.

RLHF: подкрепление по оценкам людей

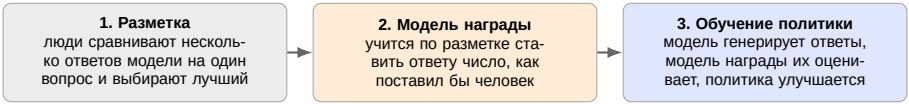


Схема описана в работе InstructGPT (Оуян и соавторы, 2022): сначала дообучение на примерах, затем модель награды, затем подкрепление. В классическом подкреплении награду задаёт заранее записанная функция. В RLHF награда выучена из предпочтений людей.

Так модели становятся вежливыми, держат диалог и реже дают вредные ответы.

Главное

RLHF превращает предпочтения людей в модель награды и учит языковую модель угождать этой оценке.

PPO: актор и критик

PPO (Шульман и соавторы, OpenAI, 2017) есть основной алгоритм RLHF.

Актор: сама модель, политика π_θ.

Критик: модель ценности, оценивает, какую награду стоит ожидать в данном состоянии.

Преимущество A_t: насколько фактическая награда лучше ожидаемой. Награда 5 хороша, если обычно 1–2, и плоха, если обычно 10–15.

Потеря актора с ограничением шага:

$$L = \mathbb{E}\Big[\min\big(\rho_t A_t,\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)A_t\big)\Big],$$

где $\rho_t = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$.

Ограничение clip не даёт новой политике уйти далеко от старой за один шаг.

Минус: критика тоже надо обучать, это дорого и медленно.

Главное

Критик задаёт ожидание, преимущество говорит, лучше или хуже ожидания получилось. Clip делает шаги осторожными.

GRPO и DPO: обойтись без лишних моделей

GRPO: без критика

Group Relative Policy Optimization (Шао и соавторы, DeepSeek, 2024). Модель даёт группу ответов на один вопрос, каждый получает награду. Ожидание задаёт средняя награда группы:

$$A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}.$$

Ответ лучше среднего по группе поощряется, хуже наказывается.

DPO: без модели награды

Direct Preference Optimization (Рафаилов и соавторы, 2023). Пары ответов: предпочтённый y_w и отвергнутый y_l. Модель напрямую повышает вероятность y_w и понижает y_l относительно исходной модели:

$$L = -\log \sigma\Big(\beta \log \frac{\pi(y_w)}{\pi_{\text{ref}}(y_w)} - \beta \log \frac{\pi(y_l)}{\pi_{\text{ref}}(y_l)}\Big).$$

алгоритм	модель награды	критик	сколько ответов на вопрос
PPO	да	да	один
GRPO	да или правило проверки	нет	группа
DPO	нет	нет	пара из разметки

Главное

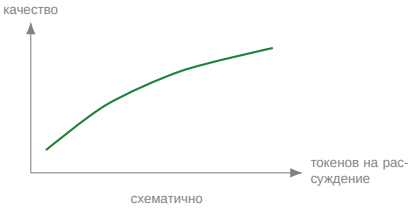
GRPO заменяет критика средним по группе, DPO заменяет подкрепление прямой оптимизацией по парам предпочтений.

Рассуждающие модели

Цепочка мыслей. Перед ответом модель пишет пошаговое рассуждение. Такие модели называют рассуждающими: o1 и o3 OpenAI, DeepSeek-R1.

Роль подкрепления. Модель получает награду за правильный ответ, к которому её привела цепочка рассуждений. Для математики и кода правильность проверяется автоматически: ответ сверяется с эталоном, код запускается тестами.

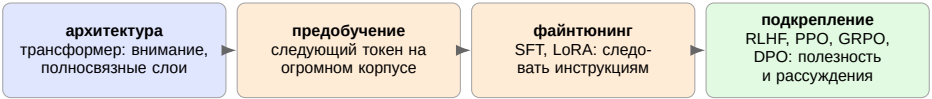
Масштабирование во время ответа (test-time scaling). Чем больше токенов модель тратит на рассуждение, тем выше качество ответа. Модель o1 показала это в 2024 году.



Главное

Рассуждения есть вторая ось роста: качество покупается вычислениями во время ответа, и обучение с подкреплением учит модель рассуждать с пользой.

Итоги



1. Модель есть функция, которая по тексту даёт вероятности следующего токена. Её веса подбирает градиентный спуск с обратным распространением.
2. Внимание связывает каждый токен со всеми другими. Цена: квадратичный рост вычислений с длиной контекста.
3. Знания приходят из предобучения, поведение из файнтюнинга и подкрепления.
4. Модели растут по двум осям: масштаб обучения и длина рассуждения при ответе.

Главное

Языковая модель знает то, что было в её корпусе, ведёт себя так, как её дообучили, и рассуждает так, как её научило подкрепление.

Источники

1. Rosenblatt F. The Perceptron. Psychological Review, 1958.
2. Rumelhart D., Hinton G., Williams R. Learning Representations by Back-Propagating Errors. Nature, 1986.
3. Hochreiter S., Schmidhuber J. Long Short-Term Memory. Neural Computation, 1997.
4. Krizhevsky A., Sutskever I., Hinton G. ImageNet Classification with Deep Convolutional Neural Networks. NeurIPS, 2012.
5. Bahdanau D., Cho K., Bengio Y. Neural Machine Translation by Jointly Learning to Align and Translate. arXiv:1409.0473, 2014.
6. Vaswani A. и др. Attention Is All You Need. NeurIPS, 2017.
7. Brown T. и др. Language Models are Few-Shot Learners (GPT-3). NeurIPS, 2020.
8. Kaplan J. и др. Scaling Laws for Neural Language Models. arXiv:2001.08361, 2020.
9. Hu E. и др. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021.
10. Dettmers T. и др. QLoRA. NeurIPS, 2023.
11. Schulman J. и др. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.
12. Ouyang L. и др. Training Language Models to Follow Instructions with Human Feedback (InstructGPT). NeurIPS, 2022.
13. Rafailov R. и др. Direct Preference Optimization. NeurIPS, 2023.
14. Shao Z. и др. DeepSeekMath (GRPO). arXiv:2402.03300, 2024.
15. DeepSeek-AI. DeepSeek-R1. arXiv:2501.12948, 2025.

Живые примеры: модель `Qwen/Qwen2.5-0.5B-Instruct`, программы в `llm/scripts`.